FIAP | ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

DOCUMENTO DE APRESENTAÇÃO DE PROJETO
ESG: Transparência na Cadeia de Suprimentos

Tema: ESG - Transparência na Cadeia de Suprimentos e Rastreamento de Emissões

Aluno: Renan Santa Rosa Xavier

RM: 566997

Tecnologias: Java 21, Spring Boot 3, Spring Data JPA, Flyway, Spring Security, Oracle Database e Docker

1. Introdução e Problema Global

No cenário corporativo atual, a adoção de práticas Ambientais, Sociais e de Governança tornou-se fundamental para a avaliação de riscos e cumprimento de leis internacionais. Entretanto, o maior desafio para as empresas não está apenas em reduzir suas próprias emissões diretas, mas sim em acompanhar o impacto ambiental dos seus fornecedores e parceiros comerciais.

Estudos recentes mostram que as emissões da cadeia de suprimentos, classificadas como **Âmbito 3** pelo *GHG Protocol*, chegam a ser em média **26 vezes maiores** do que as emissões diretas das próprias empresas. A falta de dados reais fornecidos pelos parceiros e o uso apenas de estimativas financeiras dificultam a transparência e geram riscos de não conformidade com normas globais, como a **IFRS S2** e a diretiva europeia **CSDDD**.

2. Proposta de Solução

Para resolver esse problema, foi desenvolvida uma **API RESTful em Spring Boot 3** voltada para o controle de fornecedores e de suas emissões. O sistema permite cadastrar parceiros comerciais, verificar se possuem certificação de direitos humanos e registrar relatórios de emissões de carbono de maneira simples, validada e auditável.

3. Arquitetura Técnica e Tecnologias Adotadas

O projeto foi dividido em camadas de Controller, Service, Repository, Model e DTO, com todo o código escrito em Português do Brasil para facilitar o entendimento:

- **Spring Boot 3 e Java 21:** Base para a construção da API REST, utilizando *Records* do Java para os objetos de entrada e saída de dados.
- **Validação e Tratamento de Erros com `ProblemDetail`:** Os dados recebidos passam por validações como `@NotBlank`, `@NotNull`, `@Positive` e `@PastOrPresent`. Caso algum dado esteja incorreto, a classe `TratadorGlobalExcecoes` usa `@RestControllerAdvice` para devolver o erro formatado no padrão `ProblemDetail`.
- **Spring Security:** Configurado para proteger todos os endpoints da aplicação com autenticação HTTP Basic e senhas criptografadas com `BCrypt`, exigindo login e senha em qualquer requisição.
- **Banco de Dados Oracle e Flyway:** Os dados são salvos no banco relacional **Oracle**. Para criar as tabelas `tb_fornecedores` e `tb_emissoes` automaticamente quando o projeto inicia, foi configurado o **Flyway** com scripts de migração SQL.

- **Docker e Docker Compose:** Foi criado um `Dockerfile` e um arquivo `docker-compose.yml` para subir a aplicação Java junto com o contêiner do banco de dados Oracle.

4. Endpoints Principais da API

Método	Endpoint REST	Descrição da Funcionalidade
POST	`/fornecedores`	Cadastra um novo fornecedor no banco de dados, validando o CNPJ e o certificado de direitos humanos.
GET	`/fornecedores`	Lista todos os fornecedores cadastrados no sistema.
GET	`/fornecedores/{id}/conformidade`	Calcula o total de carbono emitido pelo fornecedor e informa seu status atual de conformidade ESG.
POST	`/emissoes`	Registra um novo relatório de emissão de carbono para um fornecedor com status inicial `PENDENTE`.
GET	`/emissoes/{id}`	Busca os dados de um relatório de emissão específico pelo seu ID.
PUT	`/emissoes/{id}`	Atualiza os dados de uma emissão e permite alterar o status de auditoria de `PENDENTE` para `AUDITADO`.
DELETE	`/emissoes/{id}`	Invalida um relatório com erro mudando seu status para `INVALIDADO`, sem apagar o histórico do banco.

5. Conclusão

O desenvolvimento desta API aplica na prática os conceitos de backend vistos em aula para resolver um problema atual de Governança Ambiental. O projeto entrega um sistema funcional, protegido com Spring Security, integrado ao banco Oracle com Flyway e pronto para execução via Docker.

6. Testes Realizados na API

Os testes da aplicação foram realizados utilizando a classe de teste automatizado `EsgApiIntegrationTest` com JUnit 5 e MockMvc, validando a criação das tabelas pelo Flyway, as regras de segurança e todos os endpoints.

Teste Executado	Rota e Método HTTP	Código HTTP	Resultado
Acesso sem autenticação	`GET /fornecedores`	`401 Unauthorized`	**OK**
Cadastro de fornecedor	`POST /fornecedores`	`201 Created`	**OK**
Listagem de fornecedores	`GET /fornecedores`	`200 OK`	**OK**
Cadastro de emissão	`POST /emissoes`	`201 Created`	**OK**
Atualização e auditoria	`PUT /emissoes/2`	`200 OK`	**OK**
Consulta de conformidade ESG	`GET /fornecedores/1/conformidade`	`200 OK`	**OK**
Invalidação de relatório	`DELETE /emissoes/2`	`204 No Content`	**OK**
Erro de validação com ProblemDetail	`POST /emissoes`	`400 Bad Request`	**OK**

Resultado da Execução no Maven

```
[INFO] Running br.com.fiap.esg.EsgApiIntegrationTest
[INFO] FlywayExecutor : Successfully validated 2 migrations: V1__criar_tabelas_esg.sql,
V2__inserir_dados_iniciais.sql
[INFO] FlywayExecutor : Schema "PUBLIC" is up to date. Migration completed.
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.181 s
[INFO] BUILD SUCCESS
```

Retorno GET /fornecedores/1/conformidade

```
{
  "fornecedorId": 1,
  "nomeEmpresa": "EcoLogistica Global S.A.",
  "nivelCadeia": "TIER_1",
  "certificadoDireitosHumanos": true,
  "totalRelatoriosAtivos": 2,
  "totalCarbonoToneladas": 230.90,
  "statusConformidadeGlobal": "CONFORME_AUDITADO"
}
```

Retorno ProblemDetail HTTP 400

```
{
  "type": "about:blank",
  "title": "Erro de validação nos dados enviados",
  "status": 400,
  "detail": "Verifique os campos preenchidos
incorretamente",
  "instance": "/emissoes",
  "errosCampos": {
    "categoriaAmbito3": "A categoria de emissão é
obrigatória",
    "quantidadeCarbono": "A quantidade de carbono
deve ser maior que zero"
  }
}
```